Anomaly Detection Platform Adoption

Introduction

This document describes various ways in which Anomaly Detection platform can be utilised post rollout.

Scope

This platform can be adopted in various modes with different execution behaviour based on adoption mode. The adoption strategy can also be customised based on the required of the adoption modes. All the verifications for the build in any of the adoption modes would be run via Jenkins / Headspin integration.

Goals

- Maintain consistent page performance across releases.
- Raise alerts for detected anomalies.
- Maintain a history of performance of releases.

Metrics To Be Collected

- CPU
- FPS
- Memory
- Navigation
- TTFD
- TTID
- Device Disk Usage

Anomaly Report Template Constituents

- Whether the release profiling is RED | GREEN | YELLOW.
- Reasoning for the specific release profiling status.
- Improvement / Degradation wrt previous release.
- Variation from the agreed upon thresholds for each feature(CPU, FPS, TTFD, etc).
- Graphical representation of each features individually and in conjunction with other features.

Storage of Historical Data

Data can be classified into three groups **Recent, History, Ancient and Jurassic.**

Recent data is of past 10 releases.

History data is for dates between recent and 1 Year.

Ancient data is any data that is more than 1 Year but less than 2 years old.

Jurassic data is any data that is more than 2 Years old. It will either be archived or deleted.

- Store All the derived final aggregates on Jenkins Box.

- Raw data for **Recent** has to be stored.
- Data more than 2 years can be archived.

Adoption Modes

- Release Centric Adoption
- Feature Branch Sanity Adoption
- Developer Specific adoption
- Production Adoption

Release Centric Adoption

Description

This adoption would be tied to each release cycle of app and aimed at ensuring quality deliverables on performance front.

Verification Strategy

- Run Anomaly Detection Automation to check P0 pages(Home, PLP, PDP) performance for each branch merge via Jenkins.
- Collects metrics data for all the Anomaly Detection features.
- Detect Anomalies in post processing phase.
- Manifest the report of all the Anomaly detection features.
- Raise Alert on Anomaly Alert Slack Handle.
- Send Report on Anomaly Release Slack Handle.
- Tag relevant Stakeholders based on the page the anomaly has surfaced on(Engineering Managers).

Owners

Apps Core

Feature Branch Sanity Adoption

Description

Each major feature branch would have to run app vide anomaly detection with help from QA of respective Pod / dev. This step helps in early detection of issue with performance.

Verification Strategy

- Run anomaly detection automation post dev completion and before PR Merge.
- Check for any anomalies that surface.
- Fix anomalies if any.
- Attach the anomaly report along with PR for approval.

Owners

Pod EM

Feature Owner

Developer Specific Adoption

Description

The Anomaly Detection automation is available for perusal of all the FrontEnd Mobile App Developers. It is also accessible via code so the developer can create feature flow specific variations of performance tests, if need be.

Verification Strategy

- Create feature specific performance test cases if required. This should only be done for major features.
- Create feature specific automation test cases.
- Generate and compare reports.

Owners

Feature Dev

Production Adoption

Description

NA

Verification Strategy

NA

Owners

Apps Core